



本小节内容

插入排序原理解析

插入排序代码实战

插入排序原理解析

插入排序分为

- 1、直接插入排序
- 2、折半插入排序
- 3、希尔排序

以上 3 种插入类型的排序，考研都是考选择题，考大题概率很低，因此我们仅讲解直接插入排序的原理与代码实战，折半插入排序与希尔排序原理可以在后面的 408 课程中进行学习。

如果一个序列只有一个数，那么该序列自然是有序的。插入排序首先将第一个数视为有序序列，然后把后面 9 个数视为要依次插入的序列。首先，我们通过外层循环控制要插入的数，用 `insertVal` 保存要插入的值 87，我们比较 `arr[0]` 是否大于 `arr[1]`，即 3 是否大于 87，由于不大于，因此不发生移动，这时有序序列是 3, 87。接着，将数值 2 插入有序序列，首先将 2 赋给 `insertVal`，这时判断 87 是否大于 2，因为 87 大于 2，所以将 87 向后移动，将 2 覆盖，然后判断 3 是否大于 2，因为 3 大于 2，所以 3 移动到 87 所在的位置，内层循环结束，这时将 2 赋给 `arr[0]` 的位置，得到下表中第 2 次插入后的效果。

表 插入排序过程

	有序序列	要插入的数的序列
最初	3	87 2 93 78 56 61 38 12 40
第 1 次插入	3 87	2 93 78 56 61 38 12 40
第 2 次插入	2 3 87	93 78 56 61 38 12 40
.....		

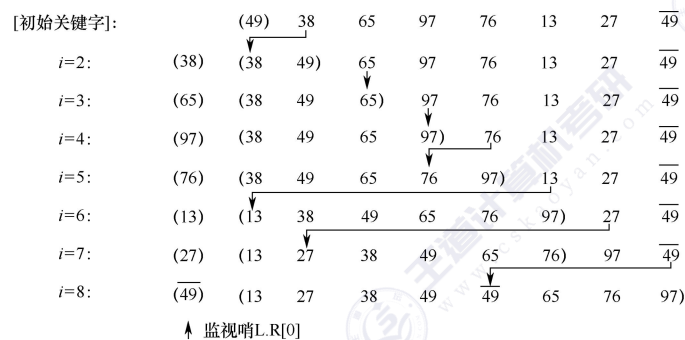
继续循环会将数依次插入有序序列，最终使得整个数组有序。插入排序主要用在部分数有序的场景，例如手机通讯录时时刻刻都是有序的，新增一个电话号码时，以插入排序的方法将其插入原有的有序序列，这样就降低了复杂度。

接下来我们借助动画网站来理解一下

<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

动画使用方法是，先点 play，然后及时点击 pause，自己通过 Step Forward 来查看

下面图是王道数据结构书插入排序用了哨兵，但是考研插入排序从没考过哨兵写法，因此掌握我们讲解的即可。



插入排序代码实战

上面已经解析了插入排序的原理，接下来我们直接进行代码实战，代码实战步骤是随机 10 个元素，然后打印，接着进行插入排序，然后打印排序后结果，代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef int ElemType;
typedef struct{
    ElemType *elem; // 整型指针
    int TableLen;
}SSTable;

void ST_Init(SSTable &ST, int len)
{
    ST.TableLen=len; // 申请 10 个元素的空间
    ST.elem=(ElemType *)malloc(sizeof(ElemType)*ST.TableLen);
    int i;
    srand(time(NULL));
    for(i=0;i<ST.TableLen;i++)
    {
        ST.elem[i]=rand()%100; // 随机了 10 个数
    }
}

void ST_print(SSTable ST)
{
    for(int i=0;i<ST.TableLen;i++)
    {
        printf("%3d", ST.elem[i]);
    }
    printf("\n");
}

// 王道书的插入排序用了哨兵，但是考研插入排序不会考哨兵，用了哨兵只是省略了 j>=0 这一句代码
void InsertSort(ElemType* arr, int n)
{

```

```
int i, j, insertVal;
for (i = 1; i < n; i++) //控制要插入的数
{
    insertVal = arr[i]; //先保存要插入的值
    //内层控制比较, j 要大于等于 0, 同时 arr[j] 大于 insertVal 时, arr[j] 位置元素往后覆盖
    for (j = i - 1; j >= 0 && arr[j] > insertVal; j--)
    {
        arr[j + 1] = arr[j];
    }
    arr[j + 1] = insertVal;
}
}
// 《王道 C 督学营》
int main()
{
    SSTable ST;
    ST_Init(ST, 10); //申请 10 个元素空间
    ST_print(ST); //排序前打印
    InsertSort(ST.elem, 10);
    ST_print(ST); //排序后再次打印
    return 0;
}
```

随着有序序列的不断增长，插入排序比较的次数也会增加，插入排序的执行次数也是从 1 加到 $N-1$ ，总运行次数为 $N(N-1)/2$ ，时间复杂度依然为 $O(n^2)$ 。因为未使用额外的空间（额外空间必须与输入元素的个数 N 相关），所以空间复杂度为 $O(1)$ 。
如果数组本身有序，那么就是最好的时间复杂度 $O(n)$ 。